

React HandsOn 19: Mocking Axios Jest GitClient

Objectives

Understanding need for isolation in testing

Understanding the concept of mocking

Using Jest for unit testing and mocking

In this hands-on lab, you will learn how to:

Unit test modules in isolation

Creating and configuring mocks and spies

Prerequisites

The following is required to complete this hands-on lab:

Node.js

NPM

Visual Studio Code

Notes

Estimated time to complete this lab: 60 minutes.

As an intern at OpenAI you are assigned the task of creating and testing a React application which will fetch and display a list of repository names for a given user.

Create a new React application using create-react-app tool and name it as "gitclientapp".

Open the application using VS Code

Go to terminal in VS Code and install the support for "axios" react library to make calls to the GitHub API.

Figure 2: Install axios library

Create a new file with the name as GitClient.js in src folder of the application

Add the following code to create a class named GitClient which will make calls to the api.github.com to fetch the repositories as follows.

Figure 3: GitClient module

Modify the App component to use the declared module to fetch and display the repositories data as follows.

Figure 4: App Component

Build and Run the application using npm start command. The output should look similar to below.

Figure 5: Application Output

Create a new unit test file named as GitClient.test.js to unit test the newly created module.

Import axios and GitClient into the unit test file.

Describe the test name as "Git Client Tests"

Create a unit test using the test() and give the test name as "should return repository names for techiesyed"

Mock the axios object to return the dummy data

Invoke the getRepositories() method of GitClient and see it's returning the mocked data instead of making an actual call to api.github.com

Run tests using npm test command.

React HandsOn 19: Mocking Axios Jest GitClient

Figure 6: Run tests